Mastering PowerShell Parameters

Advanced Techniques and Best Practices

Proper PowerShell parameter planning in the pursuit of perfection.

Jeff Hicks | https://jdhitsolutions.github.io

Bluesky	Mastodon	Github	LinkedIn
@jdhitsolutions.com	@jeffhicks	jdhitsolutions	JefferyHicks

PowerShell Parameters

- Customize the **behavior** of your PowerShell command
  - Don't assume you know all the ways someone might want to use your command
  - Write flexible code *once*.
- Avoids the use of hard-coded internal variables
- Parameters become code variables
- Not required

Parameter Planning

- Start with commands in the console
- Identify variables in your code
- Could they be something someone might want to change?
  - You can always set a default value
- How might someone use your command?
  - You can't predict all use cases
  - Don't assume the user is like you

- Proper parameters can serve as a safety check
  - Increase the chances of success

Parameter Naming

- Don't re-invent the wheel
- Names should be meaningful and predictable
  - Don't stress over length
- Avoid spaces
- Alphabet characters only!
- How can proper naming make your command easy to use?
  - How might it be used in a pipeline expression?
  - Can a parameter name match an object property name?
- How will parameter names be used by internal commands in your code?

The Parameter Attribute

```
#Import-Module PSScriptTools
Get-TypeMember System.Management.Automation.ParameterAttribute
```

```
   Type: System.Management.Automation.ParameterAttribute

Name                            MemberType ResultType       IsStatic IsEnum
----                            ---------- ----------       -------- ------
AllParameterSets                Field      System.String
GetType                         Method     Type
DontShow                        Property   Boolean
ExperimentAction                Property   ExperimentAction
ExperimentName                  Property   String
HelpMessage                     Property   String
HelpMessageBaseName             Property   String
HelpMessageResourceId           Property   String
Mandatory                       Property   Boolean
ParameterSetName                Property   String
Position                        Property   Int32
TypeId                          Property   Object
ValueFromPipeline               Property   Boolean
ValueFromPipelineByPropertyName Property   Boolean
ValueFromRemainingArguments     Property   Boolean
```

Basic Parameter Design

- Flexible input
- Function parameters might be passed to native commands

```
function Get-OS {
    [CmdletBinding()]
    param(
        $ComputerName = $env:Computername,
        $Credential,
        $Timeout = 0
    )
    begin {
            Write-Verbose "Starting $($MyInvocation.MyCommand)"
        } #begin

    process {
        try {
            #create a temp CimSession if using a credential
            if ($Credential) {
                Write-Verbose "Creating temporary CIMSession to $Computername"
                $tmpCS = New-CimSession -ComputerName $Computername -Credential $Credential
                $tmpFlag = $True
            }
            else {
                $tmpCS = $ComputerName
                $tmpFlag = $False
            }
    #...
    }

    end {
        Write-Verbose "Ending $($MyInvocation.MyCommand)"
    } #end
}
```

Position

- All parameters are positional by default
- Simple with a small number that you know will always be used

```
Function Get-Foo {
    Param($Name,$Count,$ID)

    Write-Host "Getting $count items for $name with an id of $ID"
}
```

```
Get-Foo Fred 3 123
```

```
Getting 3 items for Fred with an id of 123
```

```
Get-Command Get-Foo -Syntax
```

```
Get-Foo [[-Name] <Object>] [[-Count] <Object>] [[-ID] <Object>]
```

- More difficult for complex parameter sets
- I start at Position 0
  - Multiple parameter sets can share a position
- Use named parameters for less-used parameters
- Use named parameters for [Switch]
- Common parameters are always named

```
Function Get-Foo {
    Param(
    [Parameter(Position = 0)]
    [string]$Name,
    [Parameter(Position = 1)]
    [int]$Count,
    [int]$ID = 100
    )

    Write-Host "Getting $count items for $name with an id of $ID"
}
```

```
Get-Foo Unobtanium 100
```

```
Getting 100 items for Unobtanium with an id of 100
```

```
Get-Command Get-Foo -Syntax
```

```
Get-Foo [[-Name] <string>] [[-Count] <int>] [-ID <int>] [<CommonParameters>]
```

```
Get-Foo 'ring-of-power' 9 -id 1
```

```
Getting 9 items for ring-of-power with an id of 1
```

Alias

- Define a user-friendly parameter name
- Define an alternate parameter name for parameter binding
- Can define multiple aliases for a parameter
- Do not use the alias name as a variable name in your code

Improved Parameter Design

- Make it easy to read

```
function Get-OS {
    [CmdletBinding()]
```

```
    param(
        [Parameter(
            Position = 0,
            ValueFromPipeline,
            ValueFromPipelineByPropertyName,
            HelpMessage = 'Specify the name of a computer to query.'
        )]
        [ValidateNotNullOrEmpty()]
        [alias('CN', 'ServerName')]
        [string]$ComputerName = $env:Computername,

        [Parameter(HelpMessage = 'Specify an alternate credential')]
        [alias('RunAs')]
        [ValidateNotNullOrEmpty()]
        [PSCredential]$Credential,

        [UInt32]$Timeout = 0
    )
#...
}
```

- Pipeline input
  - Will it be a common use case?
- Parameter aliases
- Be specific about positions
  - Think about ***how*** the command will be executed
- What about validation?

Parameter Validation

- Don't assume everyone will run the command like you
- You have no way of knowing **where** a user is getting or defining a parameter value
- Set the user up for success
  - Fail first
  - Make it friendly
  - Make it useful
- Multiple parameter validations are allowed
- Casting the parameter type acts as a validation

Basic Parameters

```
param(
    $Path,
    [switch]$Recurse,
    [Switch]$IncludeFiles,
```

```
    [switch]$Hidden
)
```

With Validation

```
param(
    [Parameter(
        Position = 0,
        ValueFromPipeline,
        HelpMessage = 'Specify the root directory path to search'
    )]
    [ValidateNotNullOrEmpty()]
    [ValidateScript({ Test-Path $_ }, ErrorMessage = 'Cannot find or validate the path {0}')
    [string]$Path = '.',

    [Parameter(HelpMessage = 'Recurse through all folders.')]
    [switch]$Recurse,

    [Parameter(HelpMessage = 'Add the corresponding collection of files')]
    [alias("if")]
    [Switch]$IncludeFiles,

    [Parameter(HelpMessage = 'Include files in hidden folders')]
    [switch]$Hidden
)
```

- Create a custom error message!
  - PowerShell 7
- Use `{0}` as a value placeholder
- **Validation does not apply to default values**

```
. .\Get-FileExtensionInfo.ps1
Get-FileExtensionInfo -path c:\FooX
```

```
Get-FileExtensionInfo:
Line |
   2 |  Get-FileExtensionInfo -path c:\FooX
     |                                ~~~~~~~
     | Cannot validate argument on parameter 'Path'. Cannot find or validate the path c:\Foo
```

```
Get-FileExtensionInfo -path c:\temp


Path             : c:\temp
Extension        :
Count            : 1
PercentTotal     : 0.004
TotalSize        : 439
```

```
TotalSizePercent : 0.0001
SmallestSize     : 439
LargestSize      : 439
AverageSize      : 439
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : csv
Count            : 7
PercentTotal     : 0.0282
TotalSize        : 122819
TotalSizePercent : 0.0149
SmallestSize     : 713
LargestSize      : 80370
AverageSize      : 17545.5714285714
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : dat
Count            : 3
PercentTotal     : 0.0121
TotalSize        : 8448
TotalSizePercent : 0.001
SmallestSize     : 2816
LargestSize      : 2816
AverageSize      : 2816
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : db
Count            : 7
PercentTotal     : 0.0282
TotalSize        : 1277952
TotalSizePercent : 0.1546
```

```
SmallestSize     : 0
LargestSize      : 1073152
AverageSize      : 182564.571428571
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : dll
Count            : 1
PercentTotal     : 0.004
TotalSize        : 4096
TotalSizePercent : 0.0005
SmallestSize     : 4096
LargestSize      : 4096
AverageSize      : 4096
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : docx
Count            : 1
PercentTotal     : 0.004
TotalSize        : 19646
TotalSizePercent : 0.0024
SmallestSize     : 19646
LargestSize      : 19646
AverageSize      : 19646
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : html
Count            : 13
PercentTotal     : 0.0524
TotalSize        : 876609
TotalSizePercent : 0.1061
SmallestSize     : 2326
```

```
LargestSize      : 448224
AverageSize      : 67431.4615384615
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : jpg
Count            : 6
PercentTotal     : 0.0242
TotalSize        : 443505
TotalSizePercent : 0.0537
SmallestSize     : 2688
LargestSize      : 187257
AverageSize      : 73917.5
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : json
Count            : 134
PercentTotal     : 0.5403
TotalSize        : 157323
TotalSizePercent : 0.019
SmallestSize     : 118
LargestSize      : 55648
AverageSize      : 1174.05223880597
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : log
Count            : 1
PercentTotal     : 0.004
TotalSize        : 214
TotalSizePercent : 0
SmallestSize     : 214
LargestSize      : 214
```

```
AverageSize      : 214
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : md
Count            : 2
PercentTotal     : 0.0081
TotalSize        : 47018
TotalSizePercent : 0.0057
SmallestSize     : 5534
LargestSize      : 41484
AverageSize      : 23509
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : mscz
Count            : 2
PercentTotal     : 0.0081
TotalSize        : 985524
TotalSizePercent : 0.1192
SmallestSize     : 276283
LargestSize      : 709241
AverageSize      : 492762
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : musicxml
Count            : 2
PercentTotal     : 0.0081
TotalSize        : 1005371
TotalSizePercent : 0.1216
SmallestSize     : 264801
LargestSize      : 740570
AverageSize      : 502685.5
```

```
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : pdf
Count            : 3
PercentTotal     : 0.0121
TotalSize        : 494133
TotalSizePercent : 0.0598
SmallestSize     : 48664
LargestSize      : 389504
AverageSize      : 164711
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : png
Count            : 13
PercentTotal     : 0.0524
TotalSize        : 421693
TotalSizePercent : 0.051
SmallestSize     : 9943
LargestSize      : 73428
AverageSize      : 32437.9230769231
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : ps1
Count            : 5
PercentTotal     : 0.0202
TotalSize        : 7189
TotalSizePercent : 0.0009
SmallestSize     : 61
LargestSize      : 2806
AverageSize      : 1437.8
Computername     : PROSPERO
```

```
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : ps1xml
Count            : 5
PercentTotal     : 0.0202
TotalSize        : 18975
TotalSizePercent : 0.0023
SmallestSize     : 1434
LargestSize      : 6354
AverageSize      : 3795
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : psd1
Count            : 1
PercentTotal     : 0.004
TotalSize        : 16138
TotalSizePercent : 0.002
SmallestSize     : 16138
LargestSize      : 16138
AverageSize      : 16138
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : txt
Count            : 18
PercentTotal     : 0.0726
TotalSize        : 548203
TotalSizePercent : 0.0663
SmallestSize     : 123
LargestSize      : 318516
AverageSize      : 30455.7222222222
Computername     : PROSPERO
Platform         : Windows
```

```
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : xlsx
Count            : 2
PercentTotal     : 0.0081
TotalSize        : 206880
TotalSizePercent : 0.025
SmallestSize     : 28919
LargestSize      : 177961
AverageSize      : 103440
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : xml
Count            : 14
PercentTotal     : 0.0565
TotalSize        : 296073
TotalSizePercent : 0.0358
SmallestSize     : 100
LargestSize      : 68252
AverageSize      : 21148.0714285714
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : yml
Count            : 1
PercentTotal     : 0.004
TotalSize        : 425
TotalSizePercent : 0.0001
SmallestSize     : 425
LargestSize      : 425
AverageSize      : 425
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
```

```
Files            :
IsLargest        : False

Path             : c:\temp
Extension        : zip
Count            : 6
PercentTotal     : 0.0242
TotalSize        : 1305915
TotalSizePercent : 0.158
SmallestSize     : 71008
LargestSize      : 584393
AverageSize      : 217652.5
Computername     : PROSPERO
Platform         : Windows
ReportDate       : 4/18/2026 11:51:33 AM
Files            :
IsLargest        : True
```

Mandatory

- Require a parameter value
- Acts as a pseudo-validation
- **Cannot have a default value**

```
param(
    [Parameter(
        Position = 0,
        Mandatory,
        ValueFromPipeline,
        ValueFromPipelineByPropertyName,
        HelpMessage = 'Specify the name of a computer to query.'
    )]
    [ValidateNotNullOrEmpty()]
    [alias('CN', 'ServerName')]
    [string]$ComputerName,

    [Parameter(HelpMessage = 'Specify an alternate credential')]
    [alias('RunAs')]
    [ValidateNotNullOrEmpty()]
    [PSCredential]$Credential,

    [UInt32]$Timeout = 0
)
```

- Recommend a help message

Common Validations

ValidateNotNullOrEmpty()

```
[ValidateNotNullOrEmpty()]$param = "foo"
$param
$param = $null
```

```
foo
MetadataError:
Line |
   3 |  $param = $null
     |  ~~~~~~~~~~~~~~
     | The variable cannot be validated because the value $null is not a valid value for the
```

ValidateRange()

```
[ValidateRange(1,10)]$Count = 3
$count
$Count = 30
```

```
3
MetadataError:
Line |
   3 |  $Count = 30
     |  ~~~~~~~~~~~
     | The variable cannot be validated because the value 30 is not a valid value for the Co
```

ValidateSet()

- Must be a static set of values
- Default parameter value must belong to the set
- Automatic tab-completion

```
[ValidateSet("John","Paul","George","Ringo")]$Name = "Paul"
Write-Host "The current user is $Name"
$Name = "jeff"
```

```
The current user is Paul
MetadataError:
Line |
   3 |  $Name = "jeff"
     |  ~~~~~~~~~~~~~~
     | The variable cannot be validated because the value jeff is not a valid value for the
```

Or you can define a custom values class

```
# Inherit your class from System.Management.Automation.IValidateSetValuesGenerator
Class FileExtension : System.Management.Automation.IValidateSetValuesGenerator {
    #there no class properties
```

```
    #the GetValidValues method has no parameters
    [string[]] GetValidValues() {
        #the script block contains the code to generate the valid values
        #I'm defining the array of valid values here
        $FileExtension = @('ps1', 'ps1xml', 'txt', 'json', 'xml', 'yml', 'zip', 'md', 'csv')
        #you must use the return keyword
        return [string[]] $FileExtension
    }
}

#Not an Enum
$fe = [fileExtension]::new()
$fe.GetValidValues()
```

```
ps1
ps1xml
txt
json
xml
yml
zip
md
csv
```

```
Function Measure-FileExtension {
    [CmdletBinding()]
    param(
        [Parameter(HelpMessage = 'Enter the path to analyze')]
        [string]$Path = '.',
        #using my custom validation set
        [ValidateSet([FileExtension],ErrorMessage = "{0} is not a valid extension")]
        [string]$Extension = 'ps1',
        [switch]$Recurse
    )

    $stats = Get-ChildItem -File -Path $Path -Filter "*.$Extension" -Recurse:$Recurse |
    Measure-Object -Property Length -Sum -Average -Maximum -Minimum

    [PSCustomObject]@{
        PSTypeName = 'FileExtensionStats'
        Path       = Convert-Path $Path
        Extension  = $Extension
        Files      = $stats.Count
        TotalSize  = $stats.Sum
        Average    = $stats.Average
    }
}
```

```
Measure-FileExtension -Extension md
Measure-FileExtension -Extension "foo"
```

```
Path      : C:\Presentations\PSSummit2026\master-powershell-parameters
Extension : md
Files     : 1
TotalSize : 1157
Average   : 1157

Measure-FileExtension:
Line |
  26 |  Measure-FileExtension -Extension "foo"
     |                                   ~~~~~
     | Cannot validate argument on parameter 'Extension'. foo is not a valid extension
```

ValidatePattern()

- Parameter value must match a regex pattern

```
Function Invoke-Test {
    [cmdletbinding()]
    Param(
        [ValidatePattern("\d{4}",ErrorMessage = "{0} does not match the expected pattern of
    )

    $value
}

Invoke-Test
Invoke-Test 12
```

```
1000
Invoke-Test:
Line |
  11 |  Invoke-Test 12
     |              ~~
     | Cannot validate argument on parameter 'Value'. 12 does not match the expected pattern
```

ValidateScript({})

- Invoke a script block
- $_ is the parameter value to validate or test
- Boolean result

```
Function Invoke-Test {
```

```
    [cmdletbinding()]
    Param(
        [ValidateScript({ (Get-Verb).verb -contains $_},ErrorMessage = "The value {0} is not
    )

    "Using verb $verb"
}
Invoke-Test
Invoke-Test jump
```

```
Using verb Get
Invoke-Test:
Line |
  10 |  Invoke-Test jump
     |               ~~~~
     | Cannot validate argument on parameter 'Verb'. The value jump is not a valid command v
```

Completers

- Add tab-completion options to a parameter
- Define an `[ArgumentCompleter]` parameter attribute
- `Register-AutoCompleter`
- Validation **not** included

```
# Simple completer
# needs to run quickly
Function Get-ProcessDetail {
    [cmdletbinding()]
    Param(
        [Parameter(Position = 0, Mandatory)]
        [ArgumentCompleter({(Get-Process).name})]
        [string]$Name
    )

    Get-Process -Name $name | Select-Object ID, Name, StartTime,
    @{Name = "RunTime"; Expression = {(Get-Date) - $_.starttime}},
    Path
}
```

Create a CompletionResult Object

```
[Parameter(
    Position = 0,
    Mandatory,
    ValueFromPipelineByPropertyName,
    HelpMessage = 'Specify the name of an event log like System.'
)]
```

```
[ValidateNotNullOrEmpty()]
[ArgumentCompleter({
    [OutputType([System.Management.Automation.CompletionResult])]
    param(
        [string]$CommandName,
        [string]$ParameterName,
        [string]$WordToComplete,
        [System.Management.Automation.Language.CommandAst]$CommandAst,
        [System.Collections.IDictionary]$FakeBoundParameters
    )
    #cache results to a global variable
    if ($global:CompletionResults.Count -eq 0) {
        $global:CompletionResults = [System.Collections.Generic.List[System.Management.Autom
        (Get-WinEvent -ListLog *$wordToComplete*).LogName.Foreach({
            # CompletionText,ListItemText,ResultType,ToolTip
            $global:CompletionResults.add($([System.Management.Automation.CompletionResult]
        })
    }
    return $global:CompletionResults
})]
[string]$LogName
```

Register an Argument Completer

- Move argument completer outside of the function
- You can register completers for any command
- No commands to view or manage registered completers

```
Function Get-ServiceStatus {
    [cmdletbinding()]
    Param([string]$Computername = $env:COMPUTERNAME)

    $p = @{
        Computername = $computername
        ClassName    = "Win32_service"
        Filter       = "StartMode ='Auto' AND State<>'Running'"
    }
    Get-CimInstance @p
}

Register-ArgumentCompleter -CommandName Get-ServiceStatus -ParameterName Computername -Scrip
    param($commandName, $parameterName, $wordToComplete, $commandAst, $fakeBoundParameter)
    #run some code to create completion results
    Get-Content c:\scripts\company.txt | Where-Object {$_ -match "\w+"}  |
    ForEach-Object {
        #New(CompletionText,ListItemText,ResultType,ToolTip)
```

```
        #Tool tip shows up with PSReadline completions
        [System.Management.Automation.CompletionResult]::new($_.Trim(), $_.Trim(), 'Paramete
    }
}
```

Do More With Type

- You aren't limited to `[String]` and `[Int]`
- Let PowerShell do the work
- Very useful with parameter binding in the pipeline

```
Function Get-LocationInfo {
    [cmdletbinding()]
    Param(
        [System.IO.DirectoryInfo]$Path = "."
        )

    Write-Host "Processing $($Path.Name)" -ForegroundColor Magenta
}
```

```
Get-LocationInfo
```

```
Processing master-powershell-parameters
```

```
Function Get-FileDetail {
    [cmdletbinding()]
    Param(
        [Parameter(Mandatory)]
        [System.IO.FileInfo]$FilePath
    )

     Write-Host "Processing $($FilePath.Name) in $($FilePath.DirectoryName)" -ForegroundColo
}
```

```
Get-FileDetail $profile
```

```
Processing Microsoft.dotnet-interactive_profile.ps1 in C:\Users\Jeff\Documents\PowerShell
```

```
function Get-ServiceDetail {
    [cmdletbinding()]
    param(
        [Parameter(Mandatory)]
        [Alias('Service')]
        [ValidateScript({$_.Name},ErrorMessage = "The specified value does not appear to be
        [System.ServiceProcess.ServiceController]$Name
    )
    Write-Host "Processing $($name.Name) which is currently $($name.Status)" -ForegroundColo
}
```

```
Get-ServiceDetail wsman
Get-ServiceDetail winrm
```

```
Get-ServiceDetail:
Line |
  12 |  Get-ServiceDetail wsman
     |                     ~~~~~
     | Cannot validate argument on parameter 'Name'. The specified value does not appear to
Processing winrm which is currently Running
```

Helping Help

SupportsWildCards

- No effect on code
- **You** need to write supporting code

```
function Get-PSTypeAccelerator {
    [CmdletBinding()]
    param (
        [Parameter(
            Position = 0,
            HelpMessage = 'The name of the type accelerator like CimClass.'
        )]
        [SupportsWildcards()] #<---
        [ValidateNotNullOrEmpty()]
        [string]$Name = "*"
    )

    $Get = ([PSObject].Assembly.GetType('System.Management.Automation.TypeAccelerators')::Ge

    $Get.GetEnumerator() | Where-Object {$_.key -like $Name} |
    ForEach-Object {
        #create a custom object to store the type name and the type itself
        [PSCustomObject]@{
            PSTypeName = 'PSTypeAccelerator'
            PSVersion  = $PSVersionTable.PSVersion
            Name       = $_.Key
            Type       = $_.Value.FullName
        }
    }
}
```

- Reflected in the help

```
PS C:\> Get-Help Get-PSTypeAccelerator -Parameter Name
```

```
-Name <string>
    The name of the type accelerator like CimClass.

    Required?                    false
    Position?                    0
    Accept pipeline input?       false
    Parameter set name           (All)
    Aliases                      None
    Dynamic?                     false
    Accept wildcard characters?  true
```

Did you see how the parameter message was used in the help?

DontShow

- Parameter exists and is usable
- Remove parameter from tab-completion
- Remove parameter from PSReadline
- Use for legacy or deprecated parameters
- Exclude with Platyps
- Recommend include `[ObsoleteAttribute]`

```
function Get-OSPeek {
    [cmdletbinding()]
    param(
        [Parameter(
            Position = 0,
            ValueFromPipeline,
            ValueFromPipelineByPropertyName
        )]
        [ValidateNotNullOrEmpty()]
        [Alias('CN')]
        [string]$Computername = $env:computername,

        [ValidateNotNullOrEmpty()]
        [PSCredential]$Credential,

        [Parameter(DontShow)]
        [ValidateSet("Csv","Json","Xml")]
        [ObsoleteAttribute("This parameter is deprecated and will be removed in the next rel
        [string]$As
    )

    begin {
        $get = { Get-CimInstance -ClassName Win32_OperatingSystem -Property Caption, Install
        $PSBoundParameters.Add('Scriptblock', $Get)
        $PSBoundParameters.Add('ErrorAction', 'Stop')
```

```
        If ($PSBoundParameters.ContainsKey("As")) {
            [void]$PSBoundParameters.Remove("As")
        }
    }

    process {
        try {
            $r = Invoke-Command @PSBoundParameters
            $out =[PSCustomObject]@{
                PSTypename   = 'OSInfo'
                Name         = $r.Caption
                Installed    = $r.InstallDate
                Computername = $r.CSName
            }
        }
        catch {
            $_
            Break
        }
        if ($As) {
            #this code is deprecated
            Switch ($As) {
                "Csv" { $out | ConvertTo-Csv}
                "Json" { $out | ConvertTo-Json}
                "Xml" { $out | ConvertTo-Xml -As Stream}
            }
        }
        else {
            $out
        }
    }
    end {
        #not used
    }
}

Get-Command Get-OSPeek -Syntax


Get-OSPeek [[-Computername] <string>] [-Credential <pscredential>] [-As <string>] [<CommonPa


Get-OSPeek -as json

WARNING: Parameter 'As' is obsolete. This parameter is deprecated and will be removed in the
{
  "Name": "Microsoft Windows 11 Pro",
```

```
  "Installed": "2024-06-16T21:30:52-04:00",
  "Computername": "PROSPERO"
}
```

PSDefaultValue

- Provide a help description
- Requires comment-based help

```
#Without the attribute
function Get-FileExtensionInfo {
    <#
    .SYNOPSIS
    Get extension information
    #>
    [cmdletbinding()]
    param(
        [Parameter(
            Position = 0,
            ValueFromPipeline,
            HelpMessage = 'Specify the root directory path to search'
        )]
        [ValidateScript({ Test-Path $_ }, ErrorMessage = 'Cannot find or validate the path
        [string]$Path = '.',

        [Parameter(HelpMessage = 'Recurse through all folders.')]
        [switch]$Recurse,

        [Parameter(HelpMessage = 'Add the corresponding collection of files')]
        [alias("if")]
        [Switch]$IncludeFiles,

        [Parameter(HelpMessage = 'Include files in hidden folders')]
        [switch]$Hidden
    )

    begin {
    } #begin

    process {
    } #process

    end {
    } #end
} #close Get-FileExtensionInfo
```

```
Get-Help Get-FileExtensionInfo -parameter Path


-Path <String>

    Required?                    false
    Position?                    1
    Default value                .
    Accept pipeline input?       true (ByValue)
    Accept wildcard characters?  false




#With the attribute
function Get-FileExtensionInfo {
    <#
    .SYNOPSIS
    Get extension information
    #>
    [cmdletbinding()]
    param(
        [Parameter(
            Position = 0,
            ValueFromPipeline,
            HelpMessage = 'Specify the root directory path to search'
        )]
        [PSDefaultValue(Help = "The current location")] #<-----
        [ValidateScript({ Test-Path $_ }, ErrorMessage = 'Cannot find or validate the path
        [string]$Path = '.',

        [Parameter(HelpMessage = 'Recurse through all folders.')]
        [switch]$Recurse,

        [Parameter(HelpMessage = 'Add the corresponding collection of files')]
        [alias("if")]
        [Switch]$IncludeFiles,

        [Parameter(HelpMessage = 'Include files in hidden folders')]
        [switch]$Hidden
    )

    begin {
    } #begin

    process {
    } #process
```

```
    end {
    } #end
} #close Get-FileExtensionInfo

Get-Help Get-FileExtensionInfo -parameter Path
```

```
-Path <String>
    <-----

    Required?                    false
    Position?                    1
    Default value                The current location
    Accept pipeline input?       true (ByValue)
    Accept wildcard characters?  false
```

Experimental

- Superduper advanced topic
- Involves more than parameters

```
help about_Experimental_Features
```

Dynamic Parameters

- Define a parameter based on a condition
- Evaluated after all other parameters
- Not easily discoverable
- Difficult to document
- More complicated to reference in your code
  - Requires `[cmdletbinding()]`
  - Requires `Begin/Process/End`
  - Requires a `Param()` block
- Recommend as an exception or last resort

```
Function New-MySession {
  [cmdletbinding()]
  Param(
    [Parameter(Position = 0, Mandatory, ValueFromPipelineByPropertyName)]
    [string]$Computername,
    [int32]$Port,
    [switch]$UseSSL
  )
```

```
  DynamicParam {
    if ($Computername -match '^DOM\d+$') {
      #define a parameter attribute object
      $attributes = New-Object System.Management.Automation.ParameterAttribute
      #set parameter properties
      $attributes.ValueFromPipelineByPropertyName = $True
      $attributes.Mandatory = $True
      $attributes.HelpMessage = "Enter an alternate credential in the form domain\username c

      #define a collection for attributes
      $attributeCollection = New-Object -Type System.Collections.ObjectModel.Collection[Syst
      $attributeCollection.Add($attributes)

      #define an alias for the parameter
      $alias = New-Object System.Management.Automation.AliasAttribute -ArgumentList "RunAs"
      $attributeCollection.Add($alias)

      #define the dynamic param
      $dynParam1 = New-Object -Type System.Management.Automation.RuntimeDefinedParameter("Cr
      $dynParam1.Value = [System.Management.Automation.PSCredential]::Empty

      #create array of dynamic parameters
      $paramDictionary = New-Object -Type System.Management.Automation.RuntimeDefinedParamet
      $paramDictionary.Add("Credential", $dynParam1)
      #use the array
      return $paramDictionary

    } #if
  } #dynamic parameter

  Process {
    Write-Host "Connecting to $computername" -ForegroundColor Cyan
    #  Enter-PSSession @PSBoundParameters
  }
}

Get-Command New-MySession -Syntax


New-MySession [-Computername] <string> [-Port <int>] [-UseSSL] [<CommonParameters>]


# a more complex example with multiple dynamic parameters
code C:\Presentations\PSSummit2026\master-powershell-parameters\new-psconnection.ps1

. .\new-psconnection.ps1
# The dynamic condition doesn't rely on other parameters
```

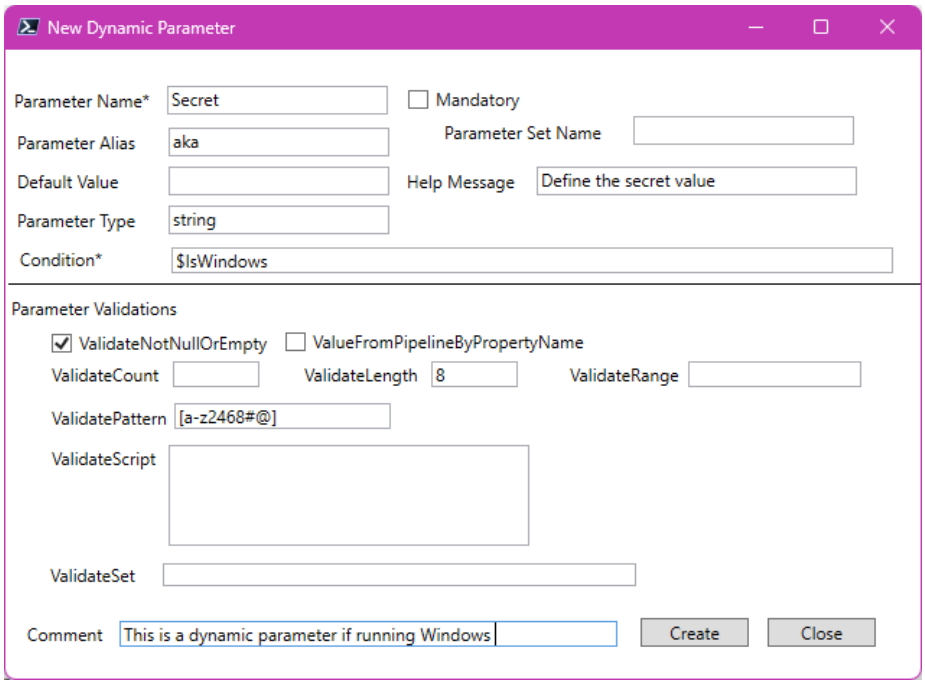
```
Get-Command New-PSConnection -syntax

New-PSConnection [-Computername] <string> [-Credential <pscredential>] [-RemoteProfile <stri

New-PSConnection -Hostname <string> [-RemoteProfile <string>] [-Username <string>] [<CommonP
```

New-DynamicParameterForm

- Part of PSScriptTools
- `New-PSDynamicParameter` function
- `New-PSDynamicParameterForm` will be integrated into ISE of VSCode
  - Import the module into VSCode
  - PowerShell: Show additional commands
  - Define a dynamic parameter
  - fill in the form
  - Click Create
  - paste into your code




```
DynamicParam {
    # This is a dynamic parameter if running Windows
    If ($IsWindows) {

    $paramDictionary = New-Object -Type System.Management.Automation.RuntimeDefinedParameter
```

```
    # Defining parameter attributes
    $attributeCollection = New-Object -Type System.Collections.ObjectModel.Collection[Systen
    $attributes = New-Object System.Management.Automation.ParameterAttribute
    $attributes.ParameterSetName = '__AllParameterSets'
    $attributes.HelpMessage = 'Define the secret value'

    # Adding ValidateLength parameter validation
    $value = @(8,)
    $v = New-Object System.Management.Automation.ValidateLengthAttribute($value)
    $AttributeCollection.Add($v)

    # Adding ValidatePattern parameter validation
    $value = '[a-z2468#@]'
    $v = New-Object System.Management.Automation.ValidatePatternAttribute($value)
    $AttributeCollection.Add($v)

    # Adding ValidateNotNullOrEmpty parameter validation
    $v = New-Object System.Management.Automation.ValidateNotNullOrEmptyAttribute
    $AttributeCollection.Add($v)
    $attributeCollection.Add($attributes)

    # Adding a parameter alias
    $dynalias = New-Object System.Management.Automation.AliasAttribute -ArgumentList 'aka'
    $attributeCollection.Add($dynalias)

    # Defining the runtime parameter
    $dynParam1 = New-Object -Type System.Management.Automation.RuntimeDefinedParameter('Secr
    $paramDictionary.Add('Secret', $dynParam1)

    return $paramDictionary
    } # end if
} #end DynamicParam
```

Form remains open until you close it.

Parameter Sets

- Don't force the user to jump through hoops
- Consider usage scenarios
- Parameters can belong to multiple sets
- Define a default parameter set
  - `[CmdletBinding(DefaultParameterSetName = "computer")]`
- Assign parameters to a set
  - `ParameterSetName = 'session'`

```
code .\Get-OS-parameter-set.ps1
```

```
#view parameter sets
. .\Get-OS-parameter-set.ps1
Get-Command Get-OS -Syntax
```

```
Get-OS [[-ComputerName] <string>] [-Credential <pscredential>] [-Timeout <uint>] [<CommonPar

Get-OS [[-CimSession] <CimSession>] [-Timeout <uint>] [<CommonParameters>]
```

```
$host.PrivateData.VerboseForegroundColor = "magenta"
Get-OS -verbose
```

```
VERBOSE: Starting Get-OS
VERBOSE: Creating temporary CIMSession to PROSPERO
VERBOSE: Operation '' complete.
VERBOSE: Perform operation 'Query CimInstances' with following parameters, ''queryExpressio
VERBOSE: Operation 'Query CimInstances' complete.

Name         : Microsoft Windows 11 Pro
Version      : 10.0.26200
BuildNumber  : 26200
Installed    : 6/16/2024 9:30:52 PM
Architecture : 64-bit
Computername : PROSPERO

VERBOSE: Removing temporary CIMSession
VERBOSE: Ending Get-OS
```

```
New-CimSession -ComputerName $env:COMPUTERNAME,"localhost" -SkipTestConnection
$r = Get-CimSession | Get-OS -Verbose
$r
```

```
Id           : 2
Name         : CimSession2
InstanceId   : b79c73ce-014a-44fd-80d5-f87e8e670214
ComputerName : PROSPERO
Protocol     : WSMAN

Id           : 3
Name         : CimSession3
InstanceId   : 639a9537-c883-43cb-923e-a8357eb19841
ComputerName : localhost
Protocol     : WSMAN

VERBOSE: Starting Get-OS
```

```
VERBOSE: Using an existing CIMSession to PROSPERO
VERBOSE: Perform operation 'Query CimInstances' with following parameters, ''queryExpressio
VERBOSE: Operation 'Query CimInstances' complete.
VERBOSE: Using an existing CIMSession to localhost
VERBOSE: Perform operation 'Query CimInstances' with following parameters, ''queryExpressio
VERBOSE: Operation 'Query CimInstances' complete.
VERBOSE: Ending Get-OS
Name         : Microsoft Windows 11 Pro
Version      : 10.0.26200
BuildNumber  : 26200
Installed    : 6/16/2024 9:30:52 PM
Architecture : 64-bit
Computername : PROSPERO

Name         : Microsoft Windows 11 Pro
Version      : 10.0.26200
BuildNumber  : 26200
Installed    : 6/16/2024 9:30:52 PM
Architecture : 64-bit
Computername : PROSPERO
```

```
Get-CimSession | Remove-CimSession
```

Tools

Get-Command

- Show syntax
- Dive into parameters

```
. .\Get-FileExtensionInfo
```

```
Get-Command Get-FileExtensionInfo -syntax
```

```
Get-FileExtensionInfo [[-Path] <string>] [-Recurse] [-IncludeFiles] [-Hidden] [<CommonParame
```

```
$cmd = Get-Command Get-FileExtensionInfo
$cmd.Parameters
```

```
Key                 Value
---                 -----
Path                System.Management.Automation.ParameterMetadata
Recurse             System.Management.Automation.ParameterMetadata
IncludeFiles        System.Management.Automation.ParameterMetadata
```

```
Hidden              System.Management.Automation.ParameterMetadata
Verbose             System.Management.Automation.ParameterMetadata
Debug               System.Management.Automation.ParameterMetadata
ErrorAction         System.Management.Automation.ParameterMetadata
WarningAction       System.Management.Automation.ParameterMetadata
InformationAction   System.Management.Automation.ParameterMetadata
ErrorVariable       System.Management.Automation.ParameterMetadata
WarningVariable     System.Management.Automation.ParameterMetadata
InformationVariable System.Management.Automation.ParameterMetadata
OutVariable         System.Management.Automation.ParameterMetadata
OutBuffer           System.Management.Automation.ParameterMetadata
PipelineVariable    System.Management.Automation.ParameterMetadata
```

```
$cmd.Parameters["Path"]
```

```
Name            : Path
ParameterType   : System.String
ParameterSets   : {[__AllParameterSets, System.Management.Automation.ParameterSetMetadata]}
IsDynamic       : False
Aliases         : {}
Attributes      : {System.Management.Automation.ValidateScriptAttribute, __AllParameterSets,
                  System.Management.Automation.ArgumentTypeConverterAttribute}
SwitchParameter : False
```

```
$cmd.Parameters["Path"].attributes
```

```
ErrorMessage                          ScriptBlock    TypeId
------------                          -----------    ------
Cannot find or validate the path {0}  Test-Path $_  System.Management.Automation.ValidateScr
                                                     System.Management.Automation.ParameterAt
                                                     System.Management.Automation.ArgumentTyp
```

```
#[Parameter()]
$cmd.Parameters["Path"].attributes[1]
```

```
ValueFromPipeline               : True
ParameterSetName                : __AllParameterSets
Position                        : 0
ValueFromPipelineByPropertyName : False
Mandatory                       : False
ValueFromRemainingArguments     : False
ExperimentName                  :
```

```
ExperimentAction                : None
HelpMessage                     : Specify the root directory path to search
HelpMessageBaseName             :
HelpMessageResourceId           :
DontShow                        : False
TypeId                          : System.Management.Automation.ParameterAttribute
```

Get-ParameterInfo

- Part of the `PSScriptTools` module
- Summary report grouped by parameter set
- Mandatory parameters will be highlighted

```
Import-Module PSScriptTools
Get-ParameterInfo Get-FileExtensionInfo
```

```
   ParameterSet: __AllParameterSets

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
Path                            False        0           System.String
Hidden                          False        Named       System.Management.Automation.Switch
IncludeFiles    if              False        Named       System.Management.Automation.Switch
Recurse                         False        Named       System.Management.Automation.Switch
```

```
Get-ParameterInfo Get-FileExtensionInfo -parameter Path | Select *
```

```
Name                            : Path
Aliases                         :
Mandatory                       : False
Position                        : 0
ValueFromPipeline               : True
ValueFromPipelineByPropertyName : False
Type                            : System.String
IsDynamic                       : False
ParameterSet                    : __AllParameterSets
```

```
#Works on any command
Get-ParameterInfo Get-Process
#highlighting doesn't work in this notebook
```

```
   ParameterSet: Id

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
FileVersionInfo FV,FVI          False        Named       System.Management.Automation.Switch
Id              PID             True         Named       System.Int32[]
Module                          False        Named       System.Management.Automation.Switch

   ParameterSet: IdWithUserName

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
Id              PID             True         Named       System.Int32[]
IncludeUserName                 True         Named       System.Management.Automation.Switch

   ParameterSet: InputObject

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
FileVersionInfo FV,FVI          False        Named       System.Management.Automation.Switch
InputObject                     True         Named       System.Diagnostics.Process[]
Module                          False        Named       System.Management.Automation.Switch

   ParameterSet: InputObjectWithUserName

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
IncludeUserName                 True         Named       System.Management.Automation.Switch
InputObject                     True         Named       System.Diagnostics.Process[]

   ParameterSet: Name

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
Name            ProcessName     False        0           System.String[]
FileVersionInfo FV,FVI          False        Named       System.Management.Automation.Switch
Module                          False        Named       System.Management.Automation.Switch

   ParameterSet: NameWithUserName

Name            Aliases         Mandatory    Position    Type
----            -------         ---------    --------    ----
Name            ProcessName     False        0           System.String[]
IncludeUserName                 True         Named       System.Management.Automation.Switch
```

Practices and Peeves

- Non-alpha parameter names
  - `$System_Name`
  - `$strComputer`
- Meaningless names
  - `$X`
  - `$Var`
- Use `[Switch]` not `[Bool]`
- Improper casing
  - `$name`
    - `$Name`
  - `$filepath`
    - `$FilePath`
- Parameter attributes are switches
  - ~~`Mandatory = $True`,ValueFromPipeline = $True~~
  - `Mandatory,ValueFromPipeline`

- Plan for parameter use
  - How will parameter values be passed?
  - What will reduce friction?
  - Set the user up for success
- Cast to type
- Sensible and practical names
  - Don't reinvent the wheel
- Validate parameters
- Make parameters self-documenting
- Dynamic parameters are an exception, not the rule

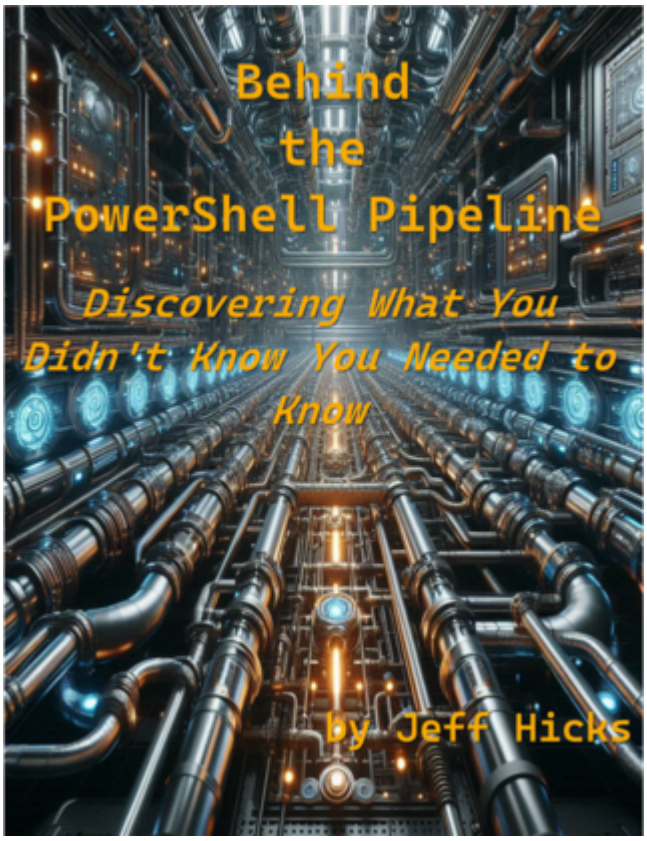
Questions

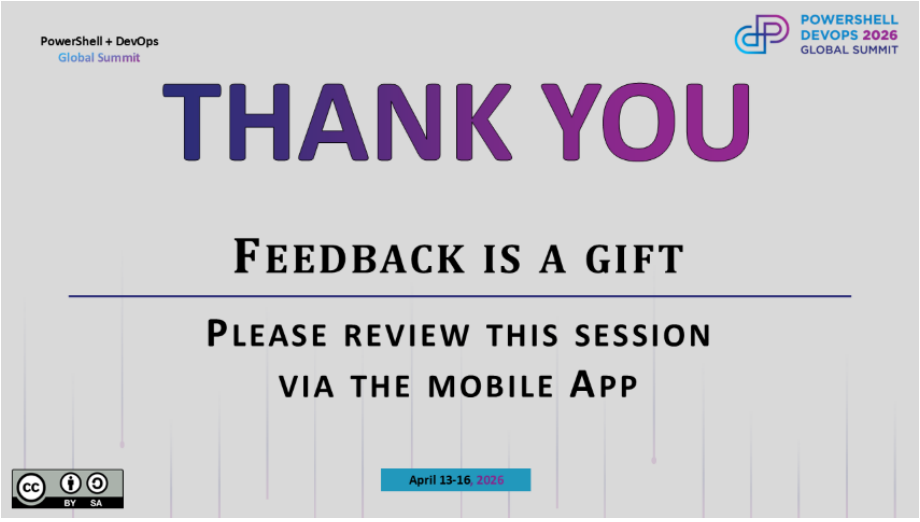
Get Help

- about_Functions_Advanced_Parameters
- about_Experimental_Features
- about_Functions_Argument_Completion

Behind the PowerShell Pipeline

- https://leanpub.com/behind-the-pspipeline


Behind
the
PowerShell Pipeline
Discovering What You
Didn't Know You Needed to
Know
by Jeff Hicks


PowerShell + DevOps
Global Summit
POWERSHELL
DEVOPS 2026
GLOBAL SUMMIT
THANK YOU
FEEDBACK IS A GIFT
PLEASE REVIEW THIS SESSION
VIA THE MOBILE APP
April 13-16, 2026